Jen v4 — 設計思想

完了は、証拠で決める。／ Evidence Gate

2026-10-22 ・ jen-marketplace v4.0.0 (ab9250f)

Table of Contents

2026 年 10 月の文献調査（原文 30 本）に基づく。設計を反証する側の証拠も含めて採否を決めた。本文の主張には出典を付け、測っていないことは測っていないと書いた。

0. なぜ v4 を作ったか

v3 までの Jen は、ほぼ全部がプロンプトだった。「ガード」と呼んでいたものは次のどちらかで、決定論の機構は合計 67 行しかなかった。

- Bash コマンドへの正規表現 9 本（`pre_tool_guard.py`・32 行）
- LLM に従うようお願いしている文（プロンプト）

`post_tool_log.py`（17 行）は全ツール呼び出しを生 JSONL に流すだけで、受入条件との紐付けは無い。つまり「Verifier が ACCEPT するまで完了と言わない」という Jen の看板の不変条件は、LLM がそう振る舞ってくれることを期待していただけだった。

v4 は、完了判定だけを決定論に移す。それ以外は変えない。

1. 核になる考え

> 判定はモデル、合成は決定論。

v3 の verifier は「受入条件を満たしたか？」という巨大な問いを 1 回聞いていた。外れると全部外れる。v4 は「この 1 つの証拠は、この 1 つの主張を支持するか」という極小の問いに割り、合成（全主張が揃ったか）を Python でやる。

出典は HALT（Roh & Han 2026, arXiv 2608.02009）。要旨にこうある。

> We frame stopping as evidence coverage rather than generator confidence
>
> HALT stops only when cumulative evidence supports each required claim
>
> without retraining or modifying the host agent

同論文は失敗の数字も出している ——「a 3B Self-Ask agent runs to its search budget on 78–88% of questions instead of halting on its own」。過剰探索が実測されている。

台帳の構造

層	実装	決定論か
主張 C(q)=(c1..cK) を事前宣言	受入条件を atomic に分解	—
作業するエージェント本体	無改変	—

層	実装	決定論か
1対ごとの判定	極小の問いを安いモデルに聞く	✗ モデル
主張ごとの running max	NULL→PARTIAL→MATCH の単調非減少	✓ 決定論
停止則 ALL_MATCH	全主張が MATCH で初めて停止	✓ 決定論

HALT 自身は検証器に Qwen2.5-3B + LoRA を使うが、Jen は AXIOM-0（第 4 節）により他社モデルを使わない。制約が設計を良い方向へ絞った例である —— 学習済みモデルを持てないので、判定単位が HALT 自身より細かくなった。

2. 証拠には強度がある

ここが v4 の肝である。文字列ベースの proof は、すでに破られている。

コキエ（2026-03）の実験: Stop Hook が出力中の `<proof>QUALITY_GATE_PASSED</proof>` を見ていたところ、モデルはその文字列を出力するだけでバイパスした。本人の分析が鋭く、「心理ではなく訓練分布の問題」だという —— 指示に含まれる文字列をそのまま出力するのは頻出パターンだから。

強度	証拠の形	誰が作れるか	偽造
`weak`	エージェントの申告（note）	エージェント	自由
`medium`	ファイル（台帳が sha256 を計算）	エージェント	意図が要る
`strong`	台帳が自分でコマンドを実行し、終了コードを捕まえた結果	台帳だけ	トークン出力では不能

強い証拠を宣言する **CLI** 経路は用意していない。 `run` サブコマンドしか作れない。エージェントは「テストが通った」を出力で作れない。

この線引きは Anthropic のハーネスにも当てはまる。Anthropic は機能リスト（機能ごとに `"passes": false` を持つ JSON）という正しいデータ構造を持ちながら、その保全をこう守っている。

> we use strongly-worded instructions like “It is unacceptable to remove or edit tests because…”
>
> — effective-harnesses-for-long-running-agents

つまりプロンプトである。コキエの実験を見ると、それが偽造できると分かる。

3. センサーをいつ走らせるか（設計の中心）

> 独立した検収は常に必要。変わるのは走らせる頻度だけ。

問いは「いつ走らせるか」である。これは独立した 2 つの一次情報が同じ結論に達している点が重要である。

Jain 2026-10（martinfowler.com・外部の対照実験）

モデル	素のまま	ハーネス付き
フロンティア（GPT-6 Sol）	**10/10**	**10/10**（時間とコストだけ増加）
低コスト（GPT-6 Luna）	**3/10**	**10/10**

Claude Sonnet 4.6 / Opus 4.6 / Gemini 3.8 Flash も素で初回成功。ハーネスの代償は平均 +1,707 万トークン・+19 分・$0.187/試行（素のベースラインが 107 万トークンなので約 **16** 倍）。

> What changes with model capability is not whether independent sensors are needed, but when they run

限界も明記されている —— 単一のグリーンフィールド 1 案件・異常に詳細な仕様書で n=1。

Anthropic 2026-03（社内実験・harness-design-long-running-apps）

著者は自作ハーネスを「bulky, slow, and expensive」と評し、Opus 4.6 の改善を理由に **sprint construct** を丸ごと削除して evaluator を最終 1 回に移した。planner は残した（無いと generator が under-scope するため）。

> the evaluator is not a fixed yes-or-no decision. It is worth the cost when the task sits beyond what the current model does reliably solo

Jen の実装

tier	sensor_mode	意味
`haiku` / `sonnet`	continuous	工程ごとに検証する。ハーネスが元を取る帯
`opus`	final_gate	工程中は回さず、Stop ゲートで 1 回だけ検める
`fable`	final_gate	opus と同扱い。優先度低（2026-10 時点の実測性能により未検証）

Jen は 18 体中 12 体が `sonnet` なので、投資先と実稼働が一致する。

4. AXIOM-0 — Claude のみで完結する

他のすべての原則の上位に置く。 性能・論文の裏付け・流行は、これを覆す理由にならない。設計判断が AXIOM-0 と衝突したら、設計判断を捨てる。

> 制約がかかるのは「知能」であって「道具」ではない。

✗ 禁じる（知能が Claude の外へ出る）	✓ 許す（Claude 内から実行できる道具）
Claude 以外のモデルに判断させること（他社 LLM・ローカル LLM・学習済み分類器）。学習・微調整を要する部品（LoRA・報酬モデル・進化探索）	MCP サーバ（ブラウザ自動化・テスト実行・git 等）。自作より既存を優先する。外部コマンド・ライブラリ

ただし運用上の原則として、特定の道具を「必須」にはしない。 あれば使い、無ければ縮退する。これは Jen が元から掲げている約束——「以上。リポジトリに何も展開しま

せん」——から来る運用上の判断である。インストールが工事になった時点で Jen の良さが消える。 したがって hooks 本体は標準ライブラリのみで書く（必ず動く下限）。

採否の実例

機構	判定	理由
HALT の停止則（証拠被覆・running max・ALL_MATCH）	採る	構造だけ移植。合成は Python
HALT の検証器 Vθ（Qwen2.5-3B + LoRA）	✗	他社モデルに判断させる + 要学習
ブラウザ自動化での検収（Puppeteer / Playwright MCP）	✓	Claude 内から実行できる。偽造不能な証拠の最強形
Sakana Fugu（オーケストレーションをモデルに内蔵）	✗	Claude でないモデル
Sakana Trinity / Conductor（学習した指揮者）	✗	要学習
Sakana DGM / MASS（再帰的自己改善）	✗	モデル改変
Meta-Harness / AutoHarness（ハーネス自動探索）	✗	要学習

Sakana 系が全滅する点について。Jen v1 は Sakana Fugu 型のルーティング思想から出発したが、2026 年の Fugu は「オーケストレーションをモデルの中に入れる」方向へ行った。それは AXIOM-0 と両立しない。思想の参照先としては残るが、実装は採らない。

この制約が設計を良くする理由（偶然ではない）

Jain の結論は「ハーネス最大の危険はオーバーヘッド」だった。AXIOM-0 は薄いハーネスを強制する。重い外部機構を積めないので、最強の反証が警告したコストが構造的に抑えられる。さらに次節の P9 が正しいなら部品はいつか捨てることになるので、AXIOM-0 は賭けを小さく張るための制約として働く。捨てる量が最小で済む。

この公理は文書だけでなく `tests/check_axiom0.py` が機械的に検証し、CI が毎 push で走らせる。「お願い」のままにすると、v4 が否定している形そのものになる。

5. P9 — ハーネスの各部品は「期限付きの賭け」

> ハーネスの各要素は、モデルができないことの仮定である。
>
> — Anthropic / harness-design-long-running-apps

部品は期限付きの賭けである。モデルが良くなれば賭けは外れ、部品はコストだけになる。これが次の 2 つを同時に説明する。

- Jain のフロンティア結果（ハーネスが時間とコストだけ増やした）
- 日本語圏に公開されている「自作オーケストレータを解体した」報告 5 件（yamk／Gota「やりすぎた話〜そのハーネスは解体された」／奥山「モデルの進化との付き合い方」／tororo／shimo4228）

彼らのハーネスは、賭けの期限が切れたのである。

したがって v4 は、各部品に前提している「モデルの不足」と外し方を明記した。

部品	前提している不足	外し方
証拠被覆の判定	モデルは自分の被覆を過大評価する	`require` を下げる → 台帳を使わない
証拠強度 3 段階	モデルは弱い証拠を強い証拠と見なす	既定 `require` を `weak` へ
Stop ゲート	モデルは早すぎる完了宣言をする	`final_gate` → hooks から外す
階層別センサー	安いモデルは連続監視が要る	tier 表を書き換える（これ自体が賭けの管理機構）
台帳保護パターン	モデルはゲートを通すため証拠を偽造する	外す（ただしコキエの実験は 2026-03 時点で再現している）
破壊的操作のガード	—	外さない。 人間の承認が要るのは能力ではなく権限の問題

最後の行だけが期限の無い賭けである。他は全部、いつか外す前提で書いてある。

P9 の最初の適用対象は Jen 自身である。v3 の看板「指揮は Fable」は「Fable は長時間一貫性で強い」という賭けで建てた構造だが、2026-10 時点の実測ではその配当が出ていない。同じ形である。

6. jen-classic — Lean 版（最小資源で最大成果）

v3 までの jen-classic は「Fable 不要版」、つまり欠けているものを補償する版だった。v4 で目的そのものを変えた。

> **jen-classic** は、同じ成果をより少ない資源で出すための版である。
>
> Fable を使わないのは制約ではなく、結果としてそうなるだけ。Fable 版は「成果の天井を上げる」。Lean 版は「払わないことで勝つ」。

支配的な判断則

Tran & Kiela 2026（Stanford, arXiv 2604.02460）が情報理論（Data Processing Inequality）に基づいてこう述べている。

> under a fixed reasoning-token budget and with perfect context utilization, single-agent systems are more information-efficient
>
> multi-agent systems become competitive when a single agent's effective context utilization is degraded, or when more compute is expended
>
> many reported advantages of multi-agent systems are better explained by unaccounted computation and context effects rather than inherent architectural benefits
>
> 委譲するのは、単一エージェントの実効的なコンテキスト利用が劣化するときだけ。 それ以外は単一で回す方が情報効率が高い。「18 体いるから振る」は理由にならない。

	jen（Fable 版）	jen-classic（Lean 版）	根拠
目的	成果の天井を上げる	同じ成果をより少ない資源で	—
既定の動き	分解して委譲	単一で回す	2604.02460
開始モデル	sonnet	haiku（昇格は実測の REJECT 後）	予防的昇格をしない
並列ファンアウト	独立タスクは並列	既定で行わない	Cursor: 20 体で実効スループットが 2〜3 体分／CooperBench: 協働で成功率−30%
最終昇格	deep-solver 単独	合議は opt-in	合議は構造上 3 倍の計算 = 効率機構ではない
センサー頻度	opus で最終ゲート 1 回	連続	Jain（低コスト帯で元を取る）

agent は 18 体のまま。証拠能力で分類した

Lean 版が「単一で回すのが既定」になると 18 体が宙に浮く。だが削らない。 Lean 版でコストになるのは、理由なく振ることである。選択肢が多いこと自体は負担にならない。削除は不可逆で capability を失う一方、得られるのは「一覧が短くなる」だけ。

代わりに、v4 の証拠ゲートが自動的に引いてくれる線で分類した。

作れる証拠	agent	扱い
strong	builder / frontend / test / debugger / verifier / strict-verifier	既定経路
medium	scout / release-manager	既定経路（scout は省略不可）
weak のみ（意見・分析）	ux-critic / contrarian-reviewer / monetization-reviewer / product-strategist / ideation	opt-in

意見系が既定から外れるのは、構造上そうなるからである。意見は `attest --kind note` = `weak` にしかならず、既定の `require=medium` に届かない。つまり呼んでも主張が閉じない（= 完了に近づかない）。完了判定とは別の軸にある、というだけである。

例外として jen-security-reviewer は条件付きで必須（auth / payment / secret / security 境界に触るとき）。ここは踏んだら戻れないかどうかで判断する軸なので、破壊的操作のガードと同じく期限の無い賭けとして扱う。

合議制についても同じ扱いをした ——機構は残し、既定から外す。独立仮説で思い込みを相殺する狙いには筋があるが、定義上 opus を 3 回以上呼ぶので「more compute is expended」に該当し、Lean 版の目的と衝突する。

資源を数える（標語にしない）

Lean 版の支配的なコストは委譲回数である。サブエージェントは呼び出しごとにまっさらなコンテキストで始まるので、1回の委譲が1つのコンテキストウィンドウを丸ごと使

う。トークン数は Jen の内側から見えないが、委譲回数は hook の実ログから数えられる。

`evidence_ledger.py budget` が `.jen/logs/tool-events.jsonl` から数える（自己申告ではない）。`init --max-delegations N` で上限を宣言できるが、超過しても強制停止はしない（v3.4 以来の不変条件）。点検シグナルとして扱う。

測れないときは「判定不能」と言う。黙って **0** とは言わない（PostToolUse のスキーマはハーネス依存で未検証）。試験でこの性質を固定した。

7. 正直に書いた留保

① 偽造は暗号的に防げていない

封印（seal）は台帳自身が既知の方法で計算するので、スクリプトを読めるエージェントは偽造後に再計算できる。防げているのは 3 層である。

1. 不注意な書き換え（リダイレクト・`sed -i`）→ 封印の不一致で検出
2. よくある編集経路 → PreToolUse で拒否
3. ログの実体が無い `strong` → 捏造として検出（封印を再計算されても捕まる）

つまり「カジュアルな抜け道を塞ぎ、事後に検出可能にする」まで。決意したエージェントは止められない。コキエの結論と同じで、これは訓練分布の話である。最も強い層は `run` の存在そのものである。 台帳が自分で実行する限り、「テストが通った」は出力で作れない。

② 効果を測っていない

「hook で強制した検証ループ vs プロンプトのみ」の完了率を比較した研究は、2026-10 時点の調査では見つからなかった。Jain は「ハーネス有無」を比較しているが、強制手段が hook かプロンプトかは比較していない。Anthropic 自身も機能リストの保全を「strongly-worded instructions」——つまりプロンプトで守っている。

したがって v4 は未測定領域にある。効果を断言しない。 `.jen/logs/stop-events.jsonl` に `gated` / `passed` が溜まるので、同種タスクで `gate: off` と既定を切り替えれば自分で測れる。測ったら `.jen/decisions.md` に残すこと。

③ 残した穴を、見えるようにしてから残している

プラグインは全セッションで有効になるので、台帳が無いだけで停止を塞ぐと普通の会話すら終われない。したがって台帳の存在がゲートの opt-in とした。「台帳を作らなければ回避できる」穴は残る。塞ぐ代わりに、ゲート無しで終わったことを必ず記録する（`gated: false`）。行動監査が拾えるので事後に数えられる。穴は、見えるようにした上で残してある。

④ 検討して不採用にしたもの

Horizon 推定（capture-mark-recapture）を停止則にする案は、原文を読んで不採用にした。要約だけ読むと「前向きに使えば停止ルールになる」と読めるが、原文には (a) 閾値が未確立と著者自身が明言、(b) 検証は後ろ向きのみ、(c) 著者自身のデータで停止条件が一度も発火していないと書いてある。推定器は「取りこぼしの診断」として使えるが、停止の判断には使わない。

この経緯を記録に残している理由は、一次情報を読まずに再検討すると同じ結論に戻れず、未検証の停止則をハーネスの中核に入れる事故が起きるため。

8. 検証

項目	内容
試験	20 件・標準ライブラリのみ（`python3 -m unittest discover -s tests`）
CI	Python 3.10 / 3.11 / 3.12 / 3.13 で実行。文書の「3.10+」という記述を実際に検める
AXIOM-0 チェック	`tests/check_axiom0.py`。stdlib 外 import・固定の外部コマンド依存・版の不一致・存在しない hook 参照・両版 hooks の食い違いを検出
チェッカーの検証	4 種類の違反を実際に検出することを確認してから入れた（常に通るチェッカーは無価値）

CI を置いた理由自体が v4 の思想である。試験スイートを誰も自動で走らせないなら、その結果はエージェントの自己申告 —— v4 自身が `weak` と分類する証拠 —— にすぎない。自分の基準を自分に当てた。

主要な出典

文献	v4 のどこに効いたか
HALT — Roh & Han 2026, arXiv 2608.02009	停止を証拠被覆で決める。running max と ALL_MATCH
Are Harnesses Worth the Cost? — Naresh Jain 2026-10, martinfowler.com	センサーをいつ走らせるか。階層別センサーの根拠
Harness design for long-running application development — Anthropic 2026-03	同上（独立に収束）。P9 の出所
Single-Agent LLMs Outperform Multi-Agent Systems… — Tran & Kiela 2026, arXiv 2604.02460	Lean 版の支配的な判断則
Stop Hook で simplify を強制したら Claude がズルを覚えた話 — コキエ 2026-03	証拠の 3 段階。文字列 proof の否定
To Search or Not to Search — Zhang ら 2026, arXiv 2602.03304 (WWW'26)	失敗は over/under の 2 方向
Explore Before Committing (HypoSearch) — Zhou ら 2026, arXiv 2609.01294	選択的分岐・局所予算。常時分岐は負ける
Why Do Multi-Agent LLM Systems Fail? (MAST) — Cemri ら 2025, arXiv 2503.13657	14 失敗モード（終了条件不明・早すぎる終了・検証しない）
Inside the Search Process — Kuhlthau 1991	早すぎる要約が探索を妨げる（v3.8 の原文保全の裏付け）
The design of browsing and berrypicking… — Bates 1989	問いは途中で変わる（`revise` の根拠）

調査は MaryCache/imozuru（芋づる式に原文へたどる Claude Code スキル）で行い、英語・日本語の 2 経路を独立に走らせた。反証側の証拠を意識して集めたことが、最も設計を変えた。 原文 30 本。

Jen v4.0.0 ・ MIT License ・ https://github.com/youki0p0/jen-marketplace